

Threat Model Report

This threat model is based on the current code in:

- `app/routes/auth.py` (`/register`, `/login`, `/logout`)
- `app/routes/tasks.py` (`/`, `/tasks/create`, `/tasks/int:task\_id/edit`, `/tasks/int:task\_id/delete`)
- `app/models/user.py` and `app/models/task.py`
- `app/__init__.py` and `config.py`

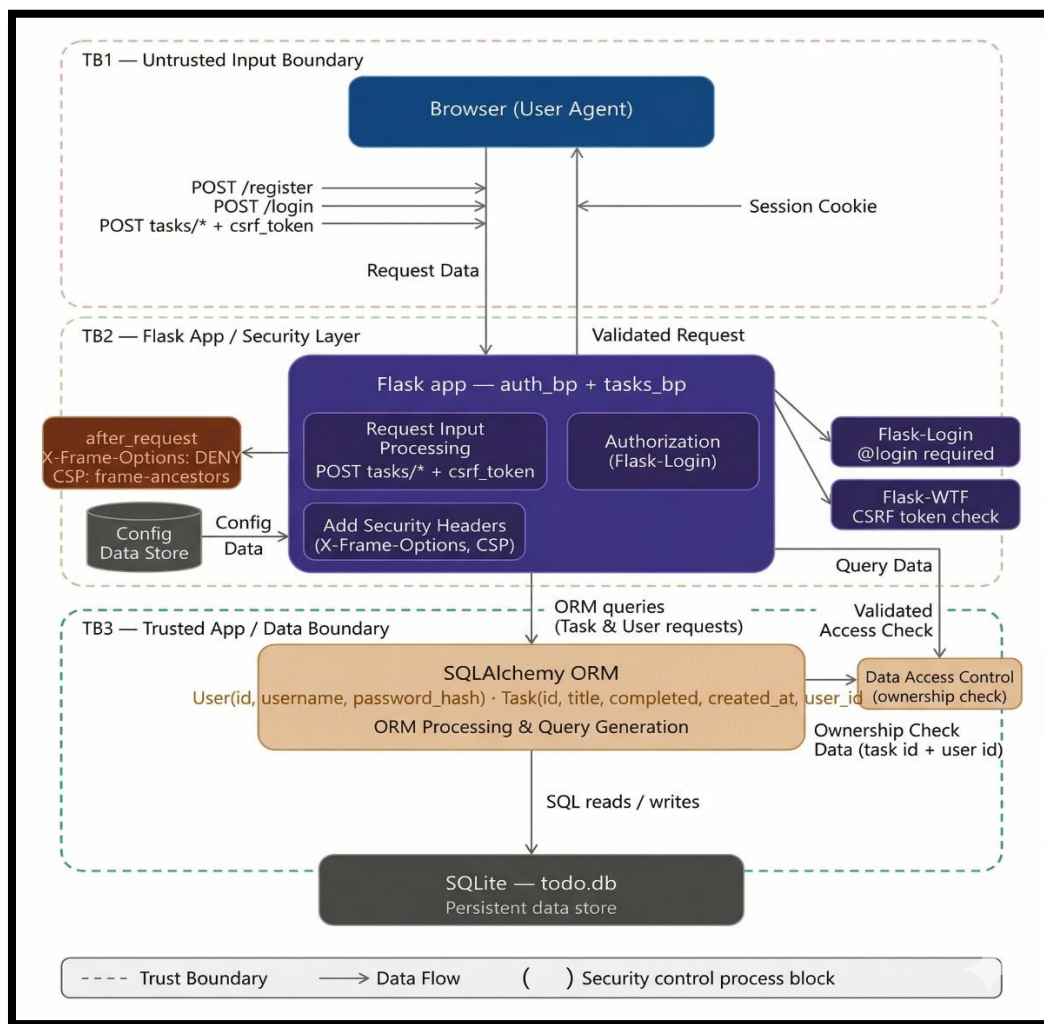
Data is stored in SQLite (`todo.db`) with `Task.user_id` as the ownership boundary.

STRIDE Threat Model:

STRIDE Category	Threat Description	Example in this To-Do App	Mitigation Status in Code
S (Spoofing)	Attacker acts as another user by stealing credentials/session.	POST to <code>/login</code> with stolen credentials then access <code>/</code> to read tasks.	Passwords hashed with <code>generate_password_hash()</code> and verified with <code>check_password_hash()</code> . Authenticated routes use <code>@login_required</code> . Gap: no lockout/rate limit on login.
T (Tampering)	User alters task data they do not own.	Change <code>task_id</code> in <code>/tasks/edit</code> or <code>/tasks/delete</code> .	Mitigated: <code>_get_user_task_or_404()</code> uses <code>Task.query.filter_by(id=task_id, user_id=current_user.id).first_or_404()</code> .
R (Repudiation)	User denies making a change; no forensic trail.	User claims they did not delete a task.	Gap: only flash messages; no persistent audit log (user id, IP, timestamp, route).
I (Information Disclosure)	Unauthorized data exposure between users or via unsafe runtime config.	View tasks by ID enumeration; stack traces if <code>debug=True</code> .	Per-user queries exist, object-level checks in place. Gap: <code>run.py</code> currently starts Flask with <code>debug=True</code> .

STRIDE Category	Threat Description	Example in this To-Do App	Mitigation Status in Code
D (Denial of Service)	Resource exhaustion blocks legitimate use.	Repeated POST flooding on /login or saturates app/DB.	Gap: no request throttling, CAPTCHA, or reverse-proxy limits in code.
E (Elevation of Privilege)	Regular user performs actions outside their scope.	Normal user attempts to access other users' tasks.	Mitigated: task ownership enforced. No admin role currently.

Data Flow Diagram (DFD):

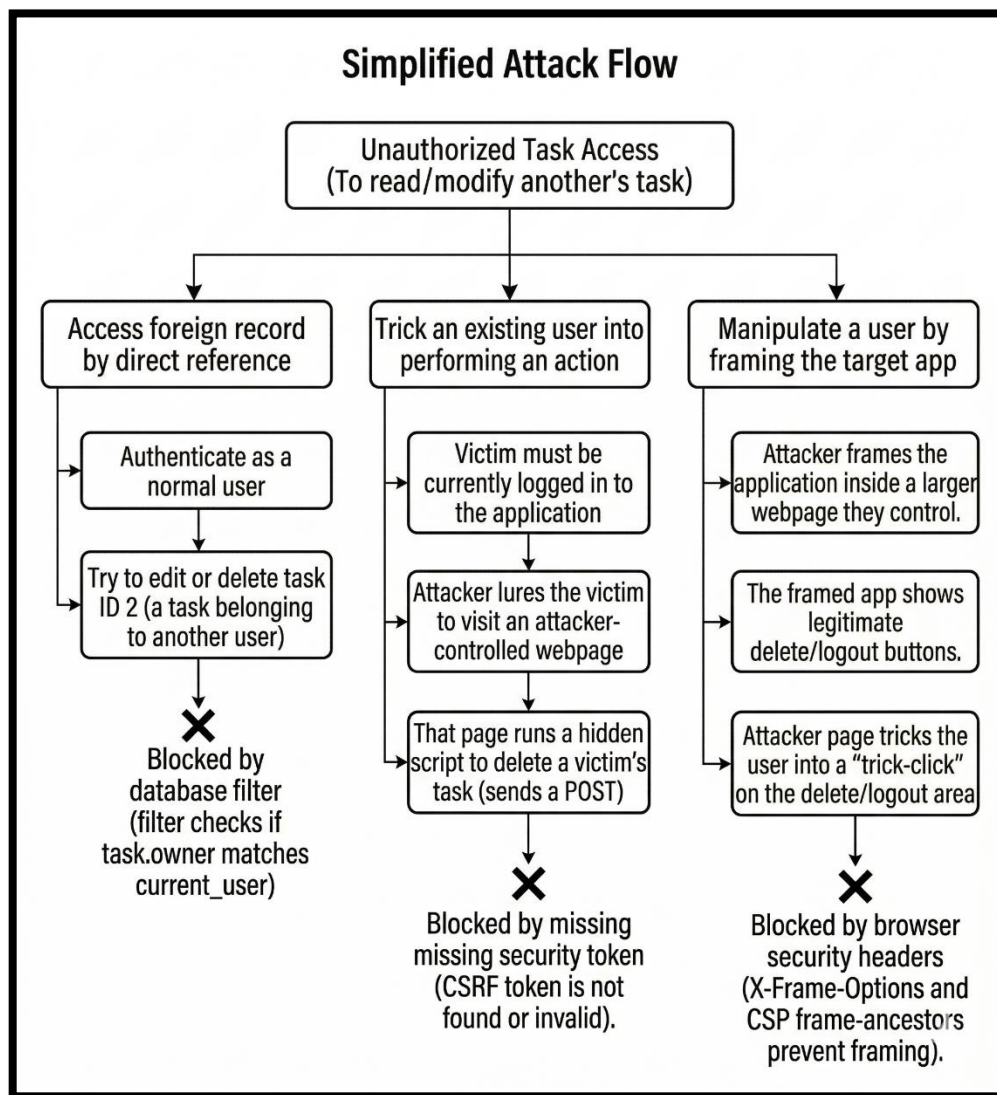


Trust Boundaries

Boundary	From	To	Nature
TB1 — Untrusted Input	Browser	Flask Application	All inputs must be validated and sanitized
TB2 — Trusted App/Data	Flask App	SQLite Database	Application-layer controls enforced before DB write
TB3 — Client Storage	Session Cookie	Browser	Client-controlled; protected by SameSite=Lax and HttpOnly

Simple Attack Tree: Unauthorized Task Access

Goal: Read/modify another user's task



Current Implemented Mitigations:

Control	Vulnerability Addressed	Implementation Detail	Status
Object-level ownership check	IDOR	<code>_get_user_task_or_404()</code> filters Task by <code>task_id</code> AND <code>current_user.id</code>	Implemented
CSRF protection	CSRF	<code>csrf.init_app(app)</code> + hidden <code>csrf_token</code> in all auth and task forms	Implemented
Logout hardening	CSRF on logout	<code>/logout</code> changed to POST method with CSRF token in navbar form	Implemented
Session cookie hardening	Session hijack / CSRF	<code>SESSION_COOKIE_SAMESITE = 'Lax'</code> set in <code>config.py</code>	Implemented
Anti-framing headers	Clickjacking	<code>app.after_request</code> sets X-Frame-Options: DENY and CSP frame-ancestors 'none'	Implemented

Residual Gaps To Track

- `run.py` uses `debug=True`; set to False in production.
- No login rate limit on `/login`.
- No audit log for task CRUD and auth events.

CVSS v3.1 Risk Ratings

Vulnerability	Base Score	Severity
IDOR (unauthorized task access)	6.5	Medium
CSRF (unauthorized form submission)	8.8	High
Clickjacking	5.4	Medium